

Jupyter Notebook Execution Report

Name: Your Name

Date: January 28, 2026

Cell 1: ■ Code

```
import pandas as pd
import numpy as np
import math
import matplotlib.pyplot as plt
```

Cell 2: ■ Code

```
df1 = pd.read_excel("MC38-d1.xlsx")
df2 = pd.read_excel("MC38-d2.xlsx")
df4 = pd.read_excel("MC38-d4.xlsx")
df5 = pd.read_excel("MC38-d5.xlsx")

df = pd.concat([df1, df2, df4, df5], ignore_index=True)
```

Cell 3: ■ Code

```
excel_file1 = pd.ExcelFile("MC38-d1.xlsx")
excel_file2 = pd.ExcelFile("MC38-d2.xlsx")
excel_file4 = pd.ExcelFile("MC38-d4.xlsx")
excel_file5 = pd.ExcelFile("MC38-d5.xlsx")

print("Day 1 excel file:", excel_file1.sheet_names)
print("Day 2 excel file:", excel_file2.sheet_names)
print("Day 4 excel file:", excel_file4.sheet_names)
print("Day 5 excel file:", excel_file5.sheet_names)
```

Output:

```
Day 1 excel file: ['plate1', 'plate2']
Day 2 excel file: ['plate1', 'plate2']
Day 4 excel file: ['plate1', 'plate2']
```

Day 5 excel file: ['plate1', 'plate2']

Cell 4: ■ Code

```
def find_all_tables_in_sheet(sheet_df, anchor("<>")):

mask = sheet_df.astype(str).applymap(lambda x: x.strip() == anchor)
positions = np.argwhere(mask.values)

if len(positions) == 0:
return []

def is_empty(x):
return pd.isna(x) or (isinstance(x, str) and x.strip() == "")

def clean_header(x):
x = str(x).strip()
try:
x = float(x)
return int(x) if x.is_integer() else x
except ValueError:
return x

tables = []

for anchor_row, anchor_col in positions:

end_col = anchor_col + 1
while end_col < sheet_df.shape[1] and not sheet_df.iloc[anchor_row + 1:,
end_col].apply(is_empty).all():
end_col += 1

end_row = anchor_row + 1
while end_row < sheet_df.shape[0] and not sheet_df.iloc[end_row, anchor_col +
1:end_col].apply(is_empty).all():
end_row += 1

extract_header = sheet_df.iloc[anchor_row, anchor_col + 1:end_col]
extract_index = sheet_df.iloc[anchor_row + 1:end_row, anchor_col]

cols = [clean_header(x) for x in extract_header]
```

```

index = extract_index.fillna("").astype(str).str.strip().tolist()

table = sheet_df.iloc[anchor_row + 1:end_row, anchor_col + 1:end_col].copy()
table.columns = cols
table.index = index

tables.append(table)

return tables

```

Cell 5: ■ Code

```

files = ["MC38-d1.xlsx", "MC38-d2.xlsx", "MC38-d4.xlsx", "MC38-d5.xlsx"]
sheet_names = ["plate1", "plate2"]

all_tables = {}

for file in files:
    sheets = pd.read_excel(file, sheet_name=sheet_names, header=None)

    for sheet in sheet_names:
        tables_in_this_sheet = find_all_tables_in_sheet(sheets[sheet], anchor="<>")

        for i, t in enumerate(tables_in_this_sheet, start=1):
            key = f"{file} | {sheet} | table{i}"
            all_tables[key] = t

print("tables found:", len(all_tables))

for name, t in all_tables.items():
    print("\n=== ", name, " ===")
    display(t)

```

Output:

```

tables found: 8
=== MC38-d1.xlsx | plate1 | table1 ===
=== MC38-d1.xlsx | plate2 | table1 ===
=== MC38-d2.xlsx | plate1 | table1 ===
=== MC38-d2.xlsx | plate2 | table1 ===
=== MC38-d4.xlsx | plate1 | table1 ===
=== MC38-d4.xlsx | plate2 | table1 ===

```

```
=== MC38-d5.xlsx | plate1 | table1 ===
```

```
=== MC38-d5.xlsx | plate2 | table1 ===
```

Index	2	3	4	5	6	7	8	9	10	11
B	139	194.0	325.0	209	293	246.0	203.0	288	293.0	364.0
C	281	399.0	140.0	244	231	193.0	56.0	55	75.0	64.0
D	151	750.0	447.0	515	439	641.0	819.0	834	182.0	123.0
E	813	136.0	501.0	329	422	325.0	811.0	579	547.0	678.0
F	-3	0.0	0.0	-5	2	-1.0	5.0	9	-5.0	6.0
G	-5	-6.0	-5.0	-8	3	-2.0	-3.0	-5	12.0	1.0

[STDERR]

```
<string>;3: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead
<string>;3: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead
<string>;3: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead
<string>;3: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead
<string>;3: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead
<string>;3: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead
<string>;3: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead
<string>;3: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead
```

Cell 6: ■ Code

```
# 1) Convert each table to long format: Row, Column, Value + Day + Plate
```

```
long_list = []
```

```
for key, t in all_tables.items():
```

```
# key looks like: "MC38-d1.xlsx | plate1 | table1"
```

```
parts = [p.strip() for p in key.split("|")]
```

```
file_name = parts[0] # MC38-d1.xlsx
```

```
plate_name = parts[1] # plate1 or plate2
```

```
# get day from filename (MC38-d1.xlsx -> d1)
```

```
day = file_name.split("-")[1].replace(".xlsx", "") # "d1", "d2", "d4", "d5"
```

```
# t is a DataFrame with:
```

```
# index = row letters (B,C,D,E, ...)
```

```
# columns = numbers (2..11)
```

```

temp = t.copy()

# make sure column labels are clean integers if possible
temp.columns = pd.to_numeric(temp.columns, errors="ignore")

# turn wide table into long table
temp_long = temp.stack().reset_index()
temp_long.columns = ["Row", "Column", "Value"]

# add day and plate columns
temp_long["Day"] = day
temp_long["Plate"] = plate_name

long_list.append(temp_long)

# combine everything
long_df = pd.concat(long_list, ignore_index=True)

# 2) Pivot so you get d1, d2, d4, d5 as separate columns (like your sample)
final_df = (
    long_df
    .pivot_table(index=["Row", "Column", "Plate"], columns="Day", values="Value",
aggfunc="first")
    .reset_index()
)

# optional: sort nicely
final_df = final_df.sort_values(["Plate", "Row", "Column"]).reset_index(drop=True)

final_df

```

Output:

	Day	Row	Column	Plate	d1	d2	d4	d5
0	B	2	plate1	114	169	146	116	
1	B	3	plate1	336.0	577.0	441.0	413.0	
2	B	4	plate1	206.0	370.0	313.0	269.0	
3	B	5	plate1	96	144	176	145	
4	B	6	plate1	266	475	415	395	
...	...	...	...	...	...	...	...	
115	G	7	plate2	NaN	-4.0	0.0	-2.0	

```

116  G      8  plate2    NaN   -9.0    8.0   -3.0
117  G      9  plate2    NaN     1     5    -5
118  G     10  plate2    NaN   -3.0    2.0   12.0
119  G     11  plate2    NaN   -1.0    0.0    1.0

```

```
[120 rows x 7 columns]
```

```
[STDERR]
```

```

<string>;19: FutureWarning: errors='ignore' is deprecated and will raise in a future version
<string>;19: FutureWarning: errors='ignore' is deprecated and will raise in a future version
<string>;19: FutureWarning: errors='ignore' is deprecated and will raise in a future version
<string>;19: FutureWarning: errors='ignore' is deprecated and will raise in a future version
<string>;19: FutureWarning: errors='ignore' is deprecated and will raise in a future version
<string>;19: FutureWarning: errors='ignore' is deprecated and will raise in a future version
<string>;19: FutureWarning: errors='ignore' is deprecated and will raise in a future version
<string>;19: FutureWarning: errors='ignore' is deprecated and will raise in a future version

```

Cell 7: ■ Code

```

pd.set_option("display.max_rows", None)
pd.set_option("display.max_columns", None)

```

Cell 8: ■ Code

```

long_list = []

for key, t in all_tables.items():
    parts = [p.strip() for p in key.split("|")]
    file_name = parts[0]
    plate_name = parts[1]

    day = file_name.split("-")[1].replace(".xlsx", "")

    temp = t.copy()
    temp.columns = pd.to_numeric(temp.columns, errors="ignore")

    temp_long = temp.stack().reset_index()
    temp_long.columns = ["Row", "Column", "Value"]

    temp_long["Day"] = day

```

```
temp_long["Plate"] = plate_name
```

```
long_list.append(temp_long)
```

```
long_df = pd.concat(long_list, ignore_index=True)
```

```
final_df = (  
    long_df  
    .pivot_table(  
        index=["Row", "Column", "Plate"],  
        columns="Day",  
        values="Value",  
        aggfunc="first"  
    )  
    .reset_index()  
)
```

```
final_df = final_df[final_df["Row"].isin(["B", "C", "D", "E"])]  
final_df = final_df.reset_index(drop=True)  
final_df = final_df.sort_values(["Plate", "Row", "Column"]).reset_index(drop=True)  
final_df["ratio d2/d1"] = final_df["d2"] / final_df["d1"]  
final_df["ratio d4/d1"] = final_df["d4"] / final_df["d1"]  
final_df["ratio d5/d1"] = final_df["d5"] / final_df["d1"]  
final_df = final_df.rename_axis(None, axis=1)
```

```
final_df
```

Output:

	Row	Column	Plate	d1	d2	d4	d5	ratio d2/d1	ratio d4/d1	\
0	B	2	plate1	114	169	146	116	1.482456	1.280702	
1	B	3	plate1	336.0	577.0	441.0	413.0	1.717262	1.3125	
2	B	4	plate1	206.0	370.0	313.0	269.0	1.796117	1.519417	
3	B	5	plate1	96	144	176	145	1.5	1.833333	
4	B	6	plate1	266	475	415	395	1.785714	1.56015	
5	B	7	plate1	135.0	259.0	286.0	225.0	1.918519	2.118519	
6	B	8	plate1	118.0	114.0	121.0	108.0	0.966102	1.025424	
7	B	9	plate1	197	323	274	269	1.639594	1.390863	
8	B	10	plate1	114.0	159.0	123.0	130.0	1.394737	1.078947	

9	B	11	plate1	188.0	413.0	381.0	352.0	2.196809	2.026596
10	C	2	plate1	118	265	237	203	2.245763	2.008475
11	C	3	plate1	113.0	209.0	218.0	153.0	1.849558	1.929204
12	C	4	plate1	180.0	317.0	238.0	210.0	1.761111	1.322222
13	C	5	plate1	181	367	326	375	2.027624	1.801105
14	C	6	plate1	209	418	382	366	2.0	1.827751
15	C	7	plate1	176.0	277.0	282.0	200.0	1.573864	1.602273
16	C	8	plate1	98.0	143.0	84.0	98.0	1.459184	0.857143
17	C	9	plate1	252	491	351	322	1.948413	1.392857
18	C	10	plate1	367.0	646.0	345.0	356.0	1.760218	0.940054
19	C	11	plate1	95.0	140.0	161.0	85.0	1.473684	1.694737
20	D	2	plate1	226	262	183	177	1.159292	0.809735
21	D	3	plate1	457.0	716.0	821.0	657.0	1.56674	1.796499
22	D	4	plate1	507.0	792.0	979.0	936.0	1.56213	1.930966
23	D	5	plate1	375	530	675	745	1.413333	1.8
24	D	6	plate1	249	379	417	446	1.522088	1.674699
25	D	7	plate1	391.0	476.0	579.0	458.0	1.217391	1.480818
26	D	8	plate1	323.0	452.0	676.0	714.0	1.399381	2.092879
27	D	9	plate1	205	305	400	423	1.487805	1.95122
28	D	10	plate1	393.0	545.0	669.0	599.0	1.386768	1.70229
29	D	11	plate1	481.0	660.0	978.0	979.0	1.372141	2.033264
30	E	2	plate1	247	369	572	626	1.493927	2.315789
31	E	3	plate1	320.0	468.0	772.0	835.0	1.4625	2.4125
32	E	4	plate1	249.0	378.0	433.0	419.0	1.518072	1.738956
33	E	5	plate1	240	301	480	483	1.254167	2.0
34	E	6	plate1	442	756	1009	1008	1.710407	2.282805
35	E	7	plate1	311.0	468.0	664.0	754.0	1.504823	2.135048
36	E	8	plate1	435.0	646.0	814.0	955.0	1.485057	1.871264
37	E	9	plate1	311	490	719	832	1.575563	2.311897
38	E	10	plate1	340.0	445.0	633.0	540.0	1.308824	1.861765
39	E	11	plate1	420.0	667.0	967.0	948.0	1.588095	2.302381
40	B	2	plate2	317	287	190	139	0.905363	0.599369
41	B	3	plate2	263.0	254.0	220.0	194.0	0.965779	0.836502
42	B	4	plate2	485.0	570.0	406.0	325.0	1.175258	0.837113

43	B	5	plate2	298	439	318	209	1.473154	1.067114
44	B	6	plate2	236	350	373	293	1.483051	1.580508
45	B	7	plate2	406.0	482.0	365.0	246.0	1.187192	0.899015
46	B	8	plate2	456.0	418.0	293.0	203.0	0.916667	0.642544
47	B	9	plate2	325	512	288	288	1.575385	0.886154
48	B	10	plate2	388.0	625.0	435.0	293.0	1.610825	1.121134
49	B	11	plate2	370.0	455.0	528.0	364.0	1.22973	1.427027
50	C	2	plate2	295	365	406	281	1.237288	1.376271
51	C	3	plate2	477.0	537.0	479.0	399.0	1.125786	1.004193
52	C	4	plate2	373.0	415.0	239.0	140.0	1.112601	0.640751
53	C	5	plate2	326	480	328	244	1.472393	1.006135
54	C	6	plate2	342	373	406	231	1.090643	1.187135
55	C	7	plate2	243.0	400.0	282.0	193.0	1.646091	1.160494
56	C	8	plate2	186.0	209.0	96.0	56.0	1.123656	0.516129
57	C	9	plate2	188	145	120	55	0.771277	0.638298
58	C	10	plate2	227.0	141.0	115.0	75.0	0.621145	0.506608
59	C	11	plate2	212.0	173.0	111.0	64.0	0.816038	0.523585
60	D	2	plate2	146	167	187	151	1.143836	1.280822
61	D	3	plate2	359.0	552.0	805.0	750.0	1.537604	2.24234
62	D	4	plate2	228.0	361.0	491.0	447.0	1.583333	2.153509
63	D	5	plate2	194	360	525	515	1.85567	2.706186
64	D	6	plate2	288	429	466	439	1.489583	1.618056
65	D	7	plate2	281.0	536.0	712.0	641.0	1.907473	2.533808
66	D	8	plate2	396.0	702.0	840.0	819.0	1.772727	2.121212
67	D	9	plate2	486	870	983	834	1.790123	2.022634
68	D	10	plate2	141.0	250.0	186.0	182.0	1.77305	1.319149
69	D	11	plate2	120.0	154.0	208.0	123.0	1.283333	1.733333
70	E	2	plate2	337	688	837	813	2.041543	2.48368
71	E	3	plate2	136.0	210.0	195.0	136.0	1.544118	1.433824
72	E	4	plate2	284.0	451.0	555.0	501.0	1.588028	1.954225
73	E	5	plate2	146	198	339	329	1.356164	2.321918
74	E	6	plate2	181	313	472	422	1.729282	2.607735
75	E	7	plate2	193.0	298.0	381.0	325.0	1.544041	1.974093
76	E	8	plate2	397.0	727.0	867.0	811.0	1.831234	2.183879

77	E	9	plate2	200	421	583	579	2.105	2.915
78	E	10	plate2	195.0	434.0	553.0	547.0	2.225641	2.835897
79	E	11	plate2	365.0	599.0	858.0	678.0	1.641096	2.350685

ratio d5/d1

0	1.017544
1	1.229167
2	1.305825
3	1.510417
4	1.484962
5	1.666667
6	0.915254
7	1.365482
8	1.140351
9	1.87234
10	1.720339
11	1.353982
12	1.166667
13	2.071823
14	1.751196
15	1.136364
16	1.0

... (74 more lines truncated)

Cell 9: ■ Code

```
#d1 plate 1 real time glow average data
#d2 plate 1 real time glow average data
#d4 plate 1 real time glow average data
#d5 plate 1 real time glow average data
# d1_average = (77+91+95)/3
# d2_average = (204+195+181)/3
# d4_average = (140+193+154)/3
# d5_average = (152+137+134)/3
# print (d1_average, d2_average, d4_average, d5_average)
```

Cell 10: ■ Code

```
long_list = []

for key, t in all_tables.items():
    parts = [p.strip() for p in key.split("|")]
    file_name = parts[0]
    plate_name = parts[1]

    day = file_name.split("-")[1].replace(".xlsx", "")

    temp = t.copy()
    temp.columns = pd.to_numeric(temp.columns, errors="ignore")

    temp_long = temp.stack().reset_index()
    temp_long.columns = ["Row", "Column", "Value"]

    temp_long["Day"] = day
    temp_long["Plate"] = plate_name

    long_list.append(temp_long)

long_df = pd.concat(long_list, ignore_index=True)
final_df = (
    long_df
    .pivot_table(
        index=["Row", "Column", "Plate"],
        columns="Day",
        values="Value",
        aggfunc="first"
    )
    .reset_index()
)

final_df = final_df[final_df["Row"].isin(["B", "C", "D", "E"])]
final_df = final_df.reset_index(drop=True)
final_df = final_df.sort_values(["Plate", "Row", "Column"]).reset_index(drop=True)
final_df = final_df.rename_axis(None, axis=1)

# subtract averages from d1, d2, d4, d5
# final_df["d1"] = final_df["d1"] - d1_average
```

```

# final_df["d2"] = final_df["d2"] - d2_average
# final_df["d4"] = final_df["d4"] - d4_average
# final_df["d5"] = final_df["d5"] - d5_average

# ratios
final_df["ratio d2/d1"] = final_df["d2"] / final_df["d1"]
final_df["ratio d4/d1"] = final_df["d4"] / final_df["d1"]
final_df["ratio d5/d1"] = final_df["d5"] / final_df["d1"]

# add column for conditions of well plate
def get_condition(row):
    plate = str(row["Plate"]).strip().lower()
    r = str(row["Row"]).strip().upper()

    if plate == "plate1":
        if r in ["B", "C"]:
            return "washer + collagen"
        if r in ["D", "E"]:
            return "cuboids only"

    if plate == "plate2":
        if r in ["B", "C"]:
            return "washer + cuboids"
        if r in ["D", "E"]:
            return "collagen only"

    return np.nan

final_df["Condition"] = final_df.apply(get_condition, axis=1)
final_df

final_df

```

Output:

	Row	Column	Plate	d1	d2	d4	d5	ratio d2/d1	ratio d4/d1	\
0	B	2	plate1	114	169	146	116	1.482456	1.280702	
1	B	3	plate1	336.0	577.0	441.0	413.0	1.717262	1.3125	
2	B	4	plate1	206.0	370.0	313.0	269.0	1.796117	1.519417	
3	B	5	plate1	96	144	176	145	1.5	1.833333	

4	B	6	plate1	266	475	415	395	1.785714	1.56015
5	B	7	plate1	135.0	259.0	286.0	225.0	1.918519	2.118519
6	B	8	plate1	118.0	114.0	121.0	108.0	0.966102	1.025424
7	B	9	plate1	197	323	274	269	1.639594	1.390863
8	B	10	plate1	114.0	159.0	123.0	130.0	1.394737	1.078947
9	B	11	plate1	188.0	413.0	381.0	352.0	2.196809	2.026596
10	C	2	plate1	118	265	237	203	2.245763	2.008475
11	C	3	plate1	113.0	209.0	218.0	153.0	1.849558	1.929204
12	C	4	plate1	180.0	317.0	238.0	210.0	1.761111	1.322222
13	C	5	plate1	181	367	326	375	2.027624	1.801105
14	C	6	plate1	209	418	382	366	2.0	1.827751
15	C	7	plate1	176.0	277.0	282.0	200.0	1.573864	1.602273
16	C	8	plate1	98.0	143.0	84.0	98.0	1.459184	0.857143
17	C	9	plate1	252	491	351	322	1.948413	1.392857
18	C	10	plate1	367.0	646.0	345.0	356.0	1.760218	0.940054
19	C	11	plate1	95.0	140.0	161.0	85.0	1.473684	1.694737
20	D	2	plate1	226	262	183	177	1.159292	0.809735
21	D	3	plate1	457.0	716.0	821.0	657.0	1.56674	1.796499
22	D	4	plate1	507.0	792.0	979.0	936.0	1.56213	1.930966
23	D	5	plate1	375	530	675	745	1.413333	1.8
24	D	6	plate1	249	379	417	446	1.522088	1.674699
25	D	7	plate1	391.0	476.0	579.0	458.0	1.217391	1.480818
26	D	8	plate1	323.0	452.0	676.0	714.0	1.399381	2.092879
27	D	9	plate1	205	305	400	423	1.487805	1.95122
28	D	10	plate1	393.0	545.0	669.0	599.0	1.386768	1.70229
29	D	11	plate1	481.0	660.0	978.0	979.0	1.372141	2.033264
30	E	2	plate1	247	369	572	626	1.493927	2.315789
31	E	3	plate1	320.0	468.0	772.0	835.0	1.4625	2.4125
32	E	4	plate1	249.0	378.0	433.0	419.0	1.518072	1.738956
33	E	5	plate1	240	301	480	483	1.254167	2.0
34	E	6	plate1	442	756	1009	1008	1.710407	2.282805
35	E	7	plate1	311.0	468.0	664.0	754.0	1.504823	2.135048
36	E	8	plate1	435.0	646.0	814.0	955.0	1.485057	1.871264
37	E	9	plate1	311	490	719	832	1.575563	2.311897

38	E	10	plate1	340.0	445.0	633.0	540.0	1.308824	1.861765
39	E	11	plate1	420.0	667.0	967.0	948.0	1.588095	2.302381
40	B	2	plate2	317	287	190	139	0.905363	0.599369
41	B	3	plate2	263.0	254.0	220.0	194.0	0.965779	0.836502
42	B	4	plate2	485.0	570.0	406.0	325.0	1.175258	0.837113
43	B	5	plate2	298	439	318	209	1.473154	1.067114
44	B	6	plate2	236	350	373	293	1.483051	1.580508
45	B	7	plate2	406.0	482.0	365.0	246.0	1.187192	0.899015
46	B	8	plate2	456.0	418.0	293.0	203.0	0.916667	0.642544
47	B	9	plate2	325	512	288	288	1.575385	0.886154
48	B	10	plate2	388.0	625.0	435.0	293.0	1.610825	1.121134
49	B	11	plate2	370.0	455.0	528.0	364.0	1.22973	1.427027
50	C	2	plate2	295	365	406	281	1.237288	1.376271
51	C	3	plate2	477.0	537.0	479.0	399.0	1.125786	1.004193
52	C	4	plate2	373.0	415.0	239.0	140.0	1.112601	0.640751
53	C	5	plate2	326	480	328	244	1.472393	1.006135
54	C	6	plate2	342	373	406	231	1.090643	1.187135
55	C	7	plate2	243.0	400.0	282.0	193.0	1.646091	1.160494
56	C	8	plate2	186.0	209.0	96.0	56.0	1.123656	0.516129
57	C	9	plate2	188	145	120	55	0.771277	0.638298
58	C	10	plate2	227.0	141.0	115.0	75.0	0.621145	0.506608
59	C	11	plate2	212.0	173.0	111.0	64.0	0.816038	0.523585
60	D	2	plate2	146	167	187	151	1.143836	1.280822
61	D	3	plate2	359.0	552.0	805.0	750.0	1.537604	2.24234
62	D	4	plate2	228.0	361.0	491.0	447.0	1.583333	2.153509
63	D	5	plate2	194	360	525	515	1.85567	2.706186
64	D	6	plate2	288	429	466	439	1.489583	1.618056
65	D	7	plate2	281.0	536.0	712.0	641.0	1.907473	2.533808
66	D	8	plate2	396.0	702.0	840.0	819.0	1.772727	2.121212
67	D	9	plate2	486	870	983	834	1.790123	2.022634
68	D	10	plate2	141.0	250.0	186.0	182.0	1.77305	1.319149
69	D	11	plate2	120.0	154.0	208.0	123.0	1.283333	1.733333
70	E	2	plate2	337	688	837	813	2.041543	2.48368
71	E	3	plate2	136.0	210.0	195.0	136.0	1.544118	1.433824

72	E	4	plate2	284.0	451.0	555.0	501.0	1.588028	1.954225
73	E	5	plate2	146	198	339	329	1.356164	2.321918
74	E	6	plate2	181	313	472	422	1.729282	2.607735
75	E	7	plate2	193.0	298.0	381.0	325.0	1.544041	1.974093
76	E	8	plate2	397.0	727.0	867.0	811.0	1.831234	2.183879
77	E	9	plate2	200	421	583	579	2.105	2.915
78	E	10	plate2	195.0	434.0	553.0	547.0	2.225641	2.835897
79	E	11	plate2	365.0	599.0	858.0	678.0	1.641096	2.350685

	ratio d5/d1	Condition
0	1.017544	washer + collagen
1	1.229167	washer + collagen
2	1.305825	washer + collagen
3	1.510417	washer + collagen
4	1.484962	washer + collagen
5	1.666667	washer + collagen
6	0.915254	washer + collagen
7	1.365482	washer + collagen
8	1.140351	washer + collagen
9	1.87234	washer + collagen
10	1.720339	washer + collagen
11	1.353982	washer + collagen
12	1.166667	washer + collagen
13	2.071823	washer + collagen
14	1.751196	washer + collagen
15	1.136364	washer + collagen
16	1.0	washer + collagen

... (74 more lines truncated)

Cell 11: ■ Code

```
final_df.to_excel("MC38_data_analysis(1).xlsx", index=False)
```

Cell 12: ■ Code

```
# Mask bad wells by setting their data to NaN, bad wells represent wells that have  
no cuboids or 2 cuboids
```

```

data_cols = [
    "d1", "d2", "d4", "d5",
    "ratio d2/d1", "ratio d4/d1", "ratio d5/d1",
    "Condition"
]

bad_wells = [
    ("plate1", "B", 3),
    # ("plate1", "C", 6),
    ("plate1", "C", 8),
    ("plate1", "C", 10),
    # ("plate1", "D", 8),
    # ("plate1", "E", 2),
    # ("plate1", "E", 7),
    # ("plate2", "C", 6),
    ("plate2", "D", 2),
    ("plate2", "D", 6),
    ("plate2", "D", 11),
    ("plate2", "E", 2),
    ("plate2", "E", 3),
    ("plate2", "E", 8),
]

```

```

for plate, row, col in bad_wells:
    mask = (
        (final_df["Plate"] == plate) &
        (final_df["Row"] == row) &
        (final_df["Column"] == col)
    )
    final_df.loc[mask, data_cols] = np.nan
final_df

```

Output:

	Row	Column	Plate	d1	d2	d4	d5	ratio d2/d1	ratio d4/d1	\
0	B	2	plate1	114	169	146	116	1.482456	1.280702	
1	B	3	plate1	NaN	NaN	NaN	NaN	NaN	NaN	
2	B	4	plate1	206.0	370.0	313.0	269.0	1.796117	1.519417	

3	B	5	plate1	96	144	176	145	1.5	1.833333
4	B	6	plate1	266	475	415	395	1.785714	1.56015
5	B	7	plate1	135.0	259.0	286.0	225.0	1.918519	2.118519
6	B	8	plate1	118.0	114.0	121.0	108.0	0.966102	1.025424
7	B	9	plate1	197	323	274	269	1.639594	1.390863
8	B	10	plate1	114.0	159.0	123.0	130.0	1.394737	1.078947
9	B	11	plate1	188.0	413.0	381.0	352.0	2.196809	2.026596
10	C	2	plate1	118	265	237	203	2.245763	2.008475
11	C	3	plate1	113.0	209.0	218.0	153.0	1.849558	1.929204
12	C	4	plate1	180.0	317.0	238.0	210.0	1.761111	1.322222
13	C	5	plate1	181	367	326	375	2.027624	1.801105
14	C	6	plate1	209	418	382	366	2.0	1.827751
15	C	7	plate1	176.0	277.0	282.0	200.0	1.573864	1.602273
16	C	8	plate1	NaN	NaN	NaN	NaN	NaN	NaN
17	C	9	plate1	252	491	351	322	1.948413	1.392857
18	C	10	plate1	NaN	NaN	NaN	NaN	NaN	NaN
19	C	11	plate1	95.0	140.0	161.0	85.0	1.473684	1.694737
20	D	2	plate1	226	262	183	177	1.159292	0.809735
21	D	3	plate1	457.0	716.0	821.0	657.0	1.56674	1.796499
22	D	4	plate1	507.0	792.0	979.0	936.0	1.56213	1.930966
23	D	5	plate1	375	530	675	745	1.413333	1.8
24	D	6	plate1	249	379	417	446	1.522088	1.674699
25	D	7	plate1	391.0	476.0	579.0	458.0	1.217391	1.480818
26	D	8	plate1	323.0	452.0	676.0	714.0	1.399381	2.092879
27	D	9	plate1	205	305	400	423	1.487805	1.95122
28	D	10	plate1	393.0	545.0	669.0	599.0	1.386768	1.70229
29	D	11	plate1	481.0	660.0	978.0	979.0	1.372141	2.033264
30	E	2	plate1	247	369	572	626	1.493927	2.315789
31	E	3	plate1	320.0	468.0	772.0	835.0	1.4625	2.4125
32	E	4	plate1	249.0	378.0	433.0	419.0	1.518072	1.738956
33	E	5	plate1	240	301	480	483	1.254167	2.0
34	E	6	plate1	442	756	1009	1008	1.710407	2.282805
35	E	7	plate1	311.0	468.0	664.0	754.0	1.504823	2.135048
36	E	8	plate1	435.0	646.0	814.0	955.0	1.485057	1.871264

37	E	9	plate1	311	490	719	832	1.575563	2.311897
38	E	10	plate1	340.0	445.0	633.0	540.0	1.308824	1.861765
39	E	11	plate1	420.0	667.0	967.0	948.0	1.588095	2.302381
40	B	2	plate2	317	287	190	139	0.905363	0.599369
41	B	3	plate2	263.0	254.0	220.0	194.0	0.965779	0.836502
42	B	4	plate2	485.0	570.0	406.0	325.0	1.175258	0.837113
43	B	5	plate2	298	439	318	209	1.473154	1.067114
44	B	6	plate2	236	350	373	293	1.483051	1.580508
45	B	7	plate2	406.0	482.0	365.0	246.0	1.187192	0.899015
46	B	8	plate2	456.0	418.0	293.0	203.0	0.916667	0.642544
47	B	9	plate2	325	512	288	288	1.575385	0.886154
48	B	10	plate2	388.0	625.0	435.0	293.0	1.610825	1.121134
49	B	11	plate2	370.0	455.0	528.0	364.0	1.22973	1.427027
50	C	2	plate2	295	365	406	281	1.237288	1.376271
51	C	3	plate2	477.0	537.0	479.0	399.0	1.125786	1.004193
52	C	4	plate2	373.0	415.0	239.0	140.0	1.112601	0.640751
53	C	5	plate2	326	480	328	244	1.472393	1.006135
54	C	6	plate2	342	373	406	231	1.090643	1.187135
55	C	7	plate2	243.0	400.0	282.0	193.0	1.646091	1.160494
56	C	8	plate2	186.0	209.0	96.0	56.0	1.123656	0.516129
57	C	9	plate2	188	145	120	55	0.771277	0.638298
58	C	10	plate2	227.0	141.0	115.0	75.0	0.621145	0.506608
59	C	11	plate2	212.0	173.0	111.0	64.0	0.816038	0.523585
60	D	2	plate2	NaN	NaN	NaN	NaN	NaN	NaN
61	D	3	plate2	359.0	552.0	805.0	750.0	1.537604	2.24234
62	D	4	plate2	228.0	361.0	491.0	447.0	1.583333	2.153509
63	D	5	plate2	194	360	525	515	1.85567	2.706186
64	D	6	plate2	NaN	NaN	NaN	NaN	NaN	NaN
65	D	7	plate2	281.0	536.0	712.0	641.0	1.907473	2.533808
66	D	8	plate2	396.0	702.0	840.0	819.0	1.772727	2.121212
67	D	9	plate2	486	870	983	834	1.790123	2.022634
68	D	10	plate2	141.0	250.0	186.0	182.0	1.77305	1.319149
69	D	11	plate2	NaN	NaN	NaN	NaN	NaN	NaN
70	E	2	plate2	NaN	NaN	NaN	NaN	NaN	NaN

71	E	3	plate2	NaN	NaN	NaN	NaN	NaN	NaN
72	E	4	plate2	284.0	451.0	555.0	501.0	1.588028	1.954225
73	E	5	plate2	146	198	339	329	1.356164	2.321918
74	E	6	plate2	181	313	472	422	1.729282	2.607735
75	E	7	plate2	193.0	298.0	381.0	325.0	1.544041	1.974093
76	E	8	plate2	NaN	NaN	NaN	NaN	NaN	NaN
77	E	9	plate2	200	421	583	579	2.105	2.915
78	E	10	plate2	195.0	434.0	553.0	547.0	2.225641	2.835897
79	E	11	plate2	365.0	599.0	858.0	678.0	1.641096	2.350685

	ratio d5/d1	Condition
0	1.017544	washer + collagen
1	NaN	NaN
2	1.305825	washer + collagen
3	1.510417	washer + collagen
4	1.484962	washer + collagen
5	1.666667	washer + collagen
6	0.915254	washer + collagen
7	1.365482	washer + collagen
8	1.140351	washer + collagen
9	1.87234	washer + collagen
10	1.720339	washer + collagen
11	1.353982	washer + collagen
12	1.166667	washer + collagen
13	2.071823	washer + collagen
14	1.751196	washer + collagen
15	1.136364	washer + collagen
16	NaN	NaN

... (64 more lines truncated)

Cell 13: ■ Code

```
final_df.to_excel("MC38_data_analysis_masked.xlsx", index=False)
```

Cell 14: ■ Code

```
conditions = [
```

```
"washer + collagen",
"cubeoids only",
"washer + cubeoids",
"collagen only"
]
```

```
day_to_ratio = {
"d1": "ratio d1/d1",
"d2": "ratio d2/d1",
"d4": "ratio d4/d1",
"d5": "ratio d5/d1",
}
```

```
tables_by_day = {}
```

```
for day, ratio_col in day_to_ratio.items():
tmp = final_df[["Condition", ratio_col]].copy()
tmp = tmp.dropna(subset=["Condition", ratio_col])
```

```
values_dict = {}
max_len = 0
```

```
for cond in conditions:
vals = tmp.loc[tmp["Condition"] == cond, ratio_col].tolist()
values_dict[cond] = vals
max_len = max(max_len, len(vals))
```

```
day_table = pd.DataFrame(index=range(1, max_len + 1), columns=conditions)
```

```
for cond in conditions:
vals = values_dict[cond]
day_table.loc[1:len(vals), cond] = vals
```

```
day_table.index.name = ratio_col
```

```
tables_by_day[day] = day_table
```

```
for day in ["d1", "d2", "d4", "d5"]:
print("\n====", day.upper(), "====")
display(tables_by_day[day])
```

Error:

Traceback (most recent call last):

```
File "/Users/jayrinxie/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 4
    exec('\n'.join(lines[:-1]), glb)
File "<string>", line 18, in <module>
File "/opt/anaconda3/lib/python3.12/site-packages/pandas/core/frame.py", line 4108, in __getitem__
    indexer = self.columns._get_indexer_strict(key, "columns")[1]
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/opt/anaconda3/lib/python3.12/site-packages/pandas/core/indexes/base.py", line 6200, in _get_indexer
    self._raise_if_missing(keyarr, indexer, axis_name)
```

```
File "/opt/anaconda3/lib/python3.12/site-packages/pandas/core/indexes/base.py", line 6252, in _raise_if_missing
    raise KeyError(f"{not_found} not in index")
```

KeyError: "['ratio d1/d1'] not in index"

Cell 15: ■ Code

```
with pd.ExcelWriter("conditions_ratios table(2).xlsx", engine="xlsxwriter") as
writer:

for day, df in tables_by_day.items():

df.to_excel(writer, sheet_name=day)
```

Error:

Traceback (most recent call last):

```
File "/Users/jayrinxie/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 4
    exec('\n'.join(lines[:-1]), glb)
File "<string>", line 2
```

```
for day, df in tables_by_day.items():
    ^
```

IndentationError: expected an indented block after 'for' statement on line 2

During handling of the above exception, another exception occurred:

Traceback (most recent call last):

```
File "/Users/jayrinxie/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 4
    exec(source, glb)
```

```
File "<string>", line 1, in <module>
```

```
File "/opt/anaconda3/lib/python3.12/site-packages/pandas/io/excel/_xlsxwriter.py", line 197, in _write_data
    from xlsxwriter import Workbook
```

ModuleNotFoundError: No module named 'xlsxwriter'

Cell 16: ■ Code

```
days = ["d1", "d2", "d4", "d5"]
```

```
conditions = [
    "washer + collagen",
    "cuboids only",
```

```

"washer + cuboids",
"collagen only"
]

tables_by_condition = {}

for cond in conditions:
    tmp = final_df.loc[final_df["Condition"] == cond, days]

    values = {}
    max_len = 0

    for day in days:
        vals = tmp[day].dropna().tolist()
        values[day] = vals
        max_len = max(max_len, len(vals))

    cond_table = pd.DataFrame(
        index=days,
        columns=range(1, max_len + 1)
    )

    for day in days:
        cond_table.loc[day, 1:len(values[day])] = values[day]

    cond_table.index.name = cond
    tables_by_condition[cond] = cond_table

for cond, df in tables_by_condition.items():
    print("\n====", cond.upper(), "====")
    display(df)

```

Output:

```

==== WASHER + COLLAGEN ====
==== CUBOIDS ONLY ====
==== WASHER + CUBOIDS ====
==== COLLAGEN ONLY ====

```

Index	1	2	3	4	5	6	7	8	9	10	11	12	13	14
d1	359.0	228.0	194	281.0	396.0	486	141.0	284.0	146	181	193.0	200	195.0	365.0

Index	1	2	3	4	5	6	7	8	9	10	11	12	13	14
d2	552.0	361.0	360	536.0	702.0	870	250.0	451.0	198	313	298.0	421	434.0	599.0
d4	805.0	491.0	525	712.0	840.0	983	186.0	555.0	339	472	381.0	583	553.0	858.0
d5	750.0	447.0	515	641.0	819.0	834	182.0	501.0	329	422	325.0	579	547.0	678.0

Cell 17: ■ Code

```
with pd.ExcelWriter("conditions_days.xlsx", engine="xlsxwriter") as writer:
    for cond in conditions:
        df = tables_by_condition[cond]
        df.to_excel(writer, sheet_name=cond)
```

Error:

```
Traceback (most recent call last):
  File "/Users/jayrinxie/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 4
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 1, in <module>
  File "/opt/anaconda3/lib/python3.12/site-packages/pandas/io/excel/_xlsxwriter.py", line 197, in
    from xlsxwriter import Workbook
ModuleNotFoundError: No module named 'xlsxwriter'
```